

Exerciții — nivelul 2

Cifre, divizori și numere prime · C++ inițiere · Vicențiu

Ce ai voie: exact ce aveai și pe fișa dinainte — `int`, `bool` · `cin`, `cout`, `endl` · `+` · `-` · `*` · `/` · `%` · comparațiile · `&&`, `||` · `if` / `else if` / `else` · `while` · funcții cu parametri și tip returnat. **În plus:** ai voie să pui un `while` în interiorul altui `while`, și ai voie să chemi o funcție de-a ta din altă funcție de-a ta.

Încă NU folosim: `for`, `do while`, vectori, `string`, `switch`, `break`, `continue`, recursivitate, `sqrt`.

Reguli: fiecare exercițiu = o funcție separată, cu semnătura din enunț · funcțiile **întorc** rezultatul, `cout` doar în `main()` · numerele citite sunt pozitive sau `0`.

Partea 1 — cifre, dar cu două bucle

`int celMaiMareNumar(int n)`

1

Întoarce cel mai mare număr care se poate forma rearanjând cifrele lui `n`.

```
celMaiMareNumar(4738) -> 8743
celMaiMareNumar(1203) -> 3210
celMaiMareNumar(7)    -> 7
```

`int celMaiMicNumar(int n)`

2

Întoarce cel mai mic număr care se poate forma rearanjând cifrele lui `n`. Zerourile ajunse în față pur și simplu dispar.

```
celMaiMicNumar(4738) -> 3478
celMaiMicNumar(4038) -> 348
celMaiMicNumar(1203) -> 123
```

`bool auAceleasiCifre(int a, int b)`

3

Întoarce `true` dacă `a` și `b` sunt formate din aceleași cifre, fiecare apărând de același număr de ori.

```
auAceleasiCifre(1223, 3212) -> true
auAceleasiCifre(4738, 8374) -> true
auAceleasiCifre(122, 1222) -> false
```

Partea 2 — divizori și numere prime

`int nrDivizori(int n)`

4

Întoarce câți divizori are `n`.

```
nrDivizori(12) -> 6
nrDivizori(7)  -> 2
nrDivizori(1)  -> 1
```

bool estePrim(int n)

5

Întoarce `true` dacă `n` este număr prim.

```
estePrim(7)  -> true
estePrim(2)  -> true
estePrim(9)  -> false
estePrim(1)  -> false
```

int sumaDivizorilorProprii(int n)

6

Întoarce suma divizorilor lui `n` mai mici strict decât `n`.

```
sumaDivizorilorProprii(12) -> 16
sumaDivizorilorProprii(7)  -> 1
sumaDivizorilorProprii(1)  -> 0
```

bool estePerfect(int n)

7

Un număr e perfect dacă e egal cu suma divizorilor lui proprii. Întoarce `true` dacă `n` e perfect.

```
estePerfect(6)  -> true
estePerfect(28) -> true
estePerfect(12) -> false
```

int cmmdc(int a, int b)

8

Întoarce cel mai mare divizor comun al numerelor `a` și `b`.

```
cmmdc(48, 18)  -> 6
cmmdc(100, 25) -> 25
cmmdc(7, 13)   -> 1
```

Partea 3 — provocări

int cmmmc(int a, int b)

9

Întoarce cel mai mic multiplu comun al numerelor `a` și `b`.

```
cmmmc(4, 6)    -> 12
cmmmc(12, 18)  -> 36
cmmmc(7, 13)   -> 91
```

bool areToateCifrelePrime(int n)

10

Întoarce `true` dacă fiecare cifră a lui `n` este, ea însăși, un număr prim.

```
areToateCifrelePrime(2357) -> true
areToateCifrelePrime(7)    -> true
areToateCifrelePrime(2347) -> false
```

`int celMaiMicNrCuSumaCifrelor(int s)`

11

Întoarce cel mai mic număr natural a cărui sumă a cifrelor este exact `s`.

```
celMaiMicNrCuSumaCifrelor(20) -> 299
celMaiMicNrCuSumaCifrelor(10) -> 19
celMaiMicNrCuSumaCifrelor(5)  -> 5
```

Numărătoarea, în `main()`

12

Citește numere de la tastatură până când tastezi `0`. La final afișează cel mai mare divizor comun al tuturor numerelor citite și câte dintre ele erau prime.

```
n = 48
n = 18
n = 30
n = 0
Cmmdc: 6
Prime: 0
```

```
n = 13
n = 7
n = 5
n = 0
Cmmdc: 1
Prime: 3
```

Înainte să-mi zici că e gata, trece fiecare funcție prin cele cinci:

- numerele `0`, `1` și `2` — fiecare separat
- un număr de o singură cifră — `7`
- un `0` pe la mijloc — `4038`
- două numere egale — `cmmdc(12, 12)`
- funcția **întoarce** rezultatul sau îl afișează?

Nu rescrie ce ai deja — cheamă funcțiile de pe fișa dinainte. Succes!